DBMS LAB · ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Operational and management reports for the airline reservation system

STUDENT
Reddy Sai Abhiram

GITHUB
github.com/reddysaiabhiram2468

PROJECT
Airline Reservation & Flight Booking System

DATABASE
MySQL 8.0 · `airline_reservation`

SUBMITTED
25 October 2026

Aim

To produce the reports an airline uses every day (passenger manifests, itineraries, load factors, route and cabin revenue, fleet utilization, payment retries and refunds) by combining tables with joins and subqueries. The queries run on the Assignment 7 data after the Assignment 8 changes, and the output is real MySQL 8.0.46 console output. The statements are saved in `sql/06_advanced_queries.sql`.

Technique	Where used
INNER JOIN across 3–6 tables	9.1, 9.2, 9.3, 9.7, 9.8, 9.10, 9.13
LEFT JOIN / anti-join	9.4, 9.5, 9.18
Self join (and the same table twice under aliases)	9.6, 9.10
Joins with views	9.2, 9.3, 9.11
Conditional aggregation, WITH ROLLUP + GROUPING(), window functions	9.8, 9.9, 9.10, 9.17
Scalar subquery	9.11, 9.14, 9.15
Nested IN subqueries	9.12
Correlated subqueries, EXISTS / NOT EXISTS	9.13, 9.14, 9.15, 9.19
Derived tables and CTEs	9.9, 9.16, 9.17

Part 1 – Join Reports

9.1 Passenger manifest for SW301 on 29 October (6 tables)

```sql
SELECT t.seat_number, c.class_name AS cabin,
       CONCAT(p.first_name, ' ', p.last_name) AS traveller,
       p.nationality, p.passport_number, b.booking_reference AS pnr, t.ticket_status
FROM ticket_details t
JOIN passengers     p ON p.passenger_id = t.passenger_id
JOIN bookings       b ON b.booking_id   = t.booking_id
JOIN travel_classes c ON c.class_code   = t.class_code
JOIN flights        f ON f.flight_id    = t.flight_id
JOIN routes         r ON r.route_id     = f.route_id
WHERE r.flight_number = 'SW301'
  AND DATE(f.departure_time) = '2026-10-29'
  AND t.ticket_status <> 'Cancelled'
ORDER BY t.class_code DESC, t.seat_number;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT t.seat_number, c.class_name AS cabin,
    ->        CONCAT(p.first_name, ' ', p.last_name) AS traveller,
    ->        p.nationality, p.passport_number, b.booking_reference AS pnr, t.ticket_status
    -> FROM ticket_details t
    -> JOIN passengers     p ON p.passenger_id = t.passenger_id
    -> JOIN bookings       b ON b.booking_id   = t.booking_id
    -> JOIN travel_classes c ON c.class_code   = t.class_code
    -> JOIN flights        f ON f.flight_id    = t.flight_id
    -> JOIN routes         r ON r.route_id     = f.route_id
    -> WHERE r.flight_number = 'SW301'
    ->   AND DATE(f.departure_time) = '2026-10-29'
    ->   AND t.ticket_status <> 'Cancelled'
    -> ORDER BY t.class_code DESC, t.seat_number;
+-------------+---------+----------------+-------------+-----------------+--------+---------------+
| seat_number | cabin   | traveller      | nationality | passport_number | pnr    | ticket_status |
+-------------+---------+----------------+-------------+-----------------+--------+---------------+
| 15F         | Economy | Ramesh Iyer    | Indian      | T8334156        | GR4GFL | Booked        |
| 16A         | Economy | Varun Reddy    | Indian      | R8677239        | R7D5DD | Booked        |
| 17A         | Economy | Geetha Iyer    | Indian      | R2706866        | GR4GFL | Booked        |
| 27E         | Economy | Harika Reddy   | Indian      | W7331886        | R7D5DD | Booked        |
| 30B         | Economy | Abhinav Gupta  | Indian      | S5091211        | XM5JYP | Booked        |
| 32B         | Economy | Lakshmi Reddy  | Indian      | W6261466        | R7D5DD | Booked        |
| 37E         | Economy | Srinivas Reddy | Indian      | S2413379        | R7D5DD | Booked        |
| 38F         | Economy | Sai Charan     | Indian      | L8535246        | BPJCWA | Booked        |
+-------------+---------+----------------+-------------+-----------------+--------+---------------+
8 rows in set (0.00 sec)
```

What it shows: This is the list handed to the cabin crew and to immigration. Because tickets record the traveller (an Assignment 3 improvement), every seat is matched to a person and a passport.

9.2 Itinerary for PNR R7D5DD: a family round trip

```sql
SELECT b.booking_reference, CONCAT(bk.first_name, ' ', bk.last_name) AS booked_by,
       b.booking_date, b.status, v.active_tickets, v.total_fare, v.amount_paid
FROM bookings b
JOIN passengers bk       ON bk.passenger_id = b.passenger_id
JOIN v_booking_totals v  ON v.booking_id    = b.booking_id
WHERE b.booking_reference = 'R7D5DD';

SELECT r.flight_number, r.origin_code AS from_, r.destination_code AS to_,
       f.departure_time, CONCAT(p.first_name, ' ', p.last_name) AS traveller,
       t.seat_number, t.fare
FROM ticket_details t
JOIN bookings   b ON b.booking_id   = t.booking_id
JOIN flights    f ON f.flight_id    = t.flight_id
JOIN routes     r ON r.route_id     = f.route_id
JOIN passengers p ON p.passenger_id = t.passenger_id
WHERE b.booking_reference = 'R7D5DD'
ORDER BY f.departure_time, t.seat_number;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT b.booking_reference, CONCAT(bk.first_name, ' ', bk.last_name) AS booked_by,
    ->        b.booking_date, b.status, v.active_tickets, v.total_fare, v.amount_paid
    -> FROM bookings b
    -> JOIN passengers bk       ON bk.passenger_id = b.passenger_id
    -> JOIN v_booking_totals v  ON v.booking_id    = b.booking_id
    -> WHERE b.booking_reference = 'R7D5DD';
+-------------------+--------------+---------------------+-----------+----------------+------------+-------------+
| booking_reference | booked_by    | booking_date        | status    | active_tickets | total_fare | amount_paid |
+-------------------+--------------+---------------------+-----------+----------------+------------+-------------+
| R7D5DD            | Lakshmi Reddy | 2026-10-11 18:37:00 | Confirmed |              8 |   27880.00 |    27880.00 |
+-------------------+--------------+---------------------+-----------+----------------+------------+-------------+
1 row in set (0.01 sec)

mysql> SELECT r.flight_number, r.origin_code AS from_, r.destination_code AS to_,
    ->        f.departure_time, CONCAT(p.first_name, ' ', p.last_name) AS traveller,
    ->        t.seat_number, t.fare
    -> FROM ticket_details t
    -> JOIN bookings   b ON b.booking_id   = t.booking_id
    -> JOIN flights    f ON f.flight_id    = t.flight_id
    -> JOIN routes     r ON r.route_id     = f.route_id
    -> JOIN passengers p ON p.passenger_id = t.passenger_id
    -> WHERE b.booking_reference = 'R7D5DD'
    -> ORDER BY f.departure_time, t.seat_number;
+---------------+-------+-----+---------------------+----------------+-------------+---------+
| flight_number | from_ | to_ | departure_time      | traveller      | seat_number | fare    |
+---------------+-------+-----+---------------------+----------------+-------------+---------+
| SW301         | HYD   | BLR | 2026-10-29 08:15:00 | Varun Reddy    | 16A         | 3470.00 |
| SW301         | HYD   | BLR | 2026-10-29 08:15:00 | Harika Reddy   | 27E         | 3470.00 |
| SW301         | HYD   | BLR | 2026-10-29 08:15:00 | Lakshmi Reddy  | 32B         | 3470.00 |
| SW301         | HYD   | BLR | 2026-10-29 08:15:00 | Srinivas Reddy | 37E         | 3470.00 |
| SW302         | BLR   | HYD | 2026-11-06 11:05:00 | Harika Reddy   | 15A         | 3500.00 |
| SW302         | BLR   | HYD | 2026-11-06 11:05:00 | Varun Reddy    | 17D         | 3500.00 |
| SW302         | BLR   | HYD | 2026-11-06 11:05:00 | Srinivas Reddy | 28F         | 3500.00 |
| SW302         | BLR   | HYD | 2026-11-06 11:05:00 | Lakshmi Reddy  | 29E         | 3500.00 |
+---------------+-------+-----+---------------------+----------------+-------------+---------+
8 rows in set (0.00 sec)
```

What it shows: The first query is the booking summary (booker, total from the view, amount paid). The second lists each traveller on each leg, which is what an e-ticket or itinerary e-mail shows.

9.3 Load factor of the ten busiest completed flights (join with a view)

```sql
SELECT o.flight_number, o.origin_code, o.destination_code, DATE(o.departure_time) AS flown_on,
       a.model, o.capacity, o.seats_sold, o.load_factor_pct
FROM v_flight_occupancy o
JOIN flights  f ON f.flight_id   = o.flight_id
JOIN aircraft a ON a.aircraft_id = f.aircraft_id
WHERE o.flight_status = 'Arrived'
ORDER BY o.seats_sold DESC, o.departure_time
LIMIT 10;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT o.flight_number, o.origin_code, o.destination_code, DATE(o.departure_time) AS flown_on,
    ->        a.model, o.capacity, o.seats_sold, o.load_factor_pct
    -> FROM v_flight_occupancy o
    -> JOIN flights  f ON f.flight_id   = o.flight_id
    -> JOIN aircraft a ON a.aircraft_id = f.aircraft_id
    -> WHERE o.flight_status = 'Arrived'
    -> ORDER BY o.seats_sold DESC, o.departure_time
    -> LIMIT 10;
+---------------+-------------+------------------+------------+----------------+----------+------------+-----------------+
| flight_number | origin_code | destination_code | flown_on   | model          | capacity | seats_sold | load_factor_pct |
+---------------+-------------+------------------+------------+----------------+----------+------------+-----------------+
| SW102         | DEL         | HYD              | 2026-10-05 | Airbus A320neo |      180 |         10 |             5.6 |
| SW601         | HYD         | GOI              | 2026-10-06 | Boeing 737-8   |      174 |         10 |             5.7 |
| SW401         | HYD         | MAA              | 2026-09-26 | Airbus A321neo |      216 |          9 |             4.2 |
| SW701         | HYD         | VTZ              | 2026-09-30 | ATR 72-600     |       70 |          8 |            11.4 |
| SW301         | HYD         | BLR              | 2026-10-01 | Boeing 737-8   |      174 |          7 |             4.0 |
| SW801         | HYD         | DXB              | 2026-10-09 | Boeing 787-9   |      253 |          7 |             2.8 |
| SW702         | VTZ         | HYD              | 2026-10-01 | ATR 72-600     |       70 |          6 |             8.6 |
| SW402         | MAA         | HYD              | 2026-10-04 | Airbus A321neo |      216 |          6 |             2.8 |
| SW201         | HYD         | BOM              | 2026-10-06 | Airbus A320neo |      180 |          6 |             3.3 |
| SW702         | VTZ         | HYD              | 2026-10-08 | ATR 72-600     |       70 |          6 |             8.6 |
+---------------+-------------+------------------+------------+----------------+----------+------------+-----------------+
10 rows in set (0.00 sec)
```

What it shows: The view counts live tickets per flight; joining aircraft adds the model. The sample contains only a subset of the airline's real bookings, so load factors are low.

9.4 Flights without a single live ticket (LEFT JOIN)

```
SELECT f.flight_id, r.flight_number, f.departure_time, f.flight_status
FROM flights f
JOIN routes r ON r.route_id = f.route_id
LEFT JOIN ticket_details t
       ON t.flight_id = f.flight_id AND t.ticket_status <> 'Cancelled'
WHERE t.ticket_id IS NULL
ORDER BY f.departure_time;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT f.flight_id, r.flight_number, f.departure_time, f.flight_status
    -> FROM flights f
    -> JOIN routes r ON r.route_id = f.route_id
    -> LEFT JOIN ticket_details t
    ->        ON t.flight_id = f.flight_id AND t.ticket_status <> 'Cancelled'
    -> WHERE t.ticket_id IS NULL
    -> ORDER BY f.departure_time;
+-----------+---------------+---------------------+---------------+
| flight_id | flight_number | departure_time      | flight_status |
+-----------+---------------+---------------------+---------------+
|        13 | SW201         | 2026-09-29 07:40:00 | Arrived       |
|        78 | SW802         | 2026-10-01 03:30:00 | Arrived       |
|        88 | SW651         | 2026-10-05 14:20:00 | Cancelled     |
|        15 | SW201         | 2026-10-13 07:40:00 | Arrived       |
|         4 | SW101         | 2026-10-18 06:10:00 | Arrived       |
|        46 | SW402         | 2026-10-18 16:10:00 | Arrived       |
|        90 | SW651         | 2026-10-19 14:20:00 | Cancelled     |
|        58 | SW601         | 2026-10-20 12:10:00 | Arrived       |
|        85 | SW901         | 2026-10-20 23:55:00 | Arrived       |
|        59 | SW601         | 2026-10-27 12:10:00 | Scheduled     |
|        71 | SW702         | 2026-10-29 19:10:00 | Scheduled     |
+-----------+---------------+---------------------+---------------+
11 rows in set (0.00 sec)
```

What it shows: The LEFT JOIN keeps every flight, and `t.ticket_id IS NULL` keeps those without a live ticket. The condition on the ticket status is placed in ON so that it does not remove the empty flights. The two cancelled SW651 flights appear because all their tickets were cancelled. The scheduled flights at the end still need to be sold.

9.5 Registered passengers who have never booked or travelled (anti-join)

```
SELECT p.passenger_id, p.first_name, p.last_name, p.email
FROM passengers p
LEFT JOIN bookings b       ON b.passenger_id = p.passenger_id
LEFT JOIN ticket_details t ON t.passenger_id = p.passenger_id
WHERE b.booking_id IS NULL AND t.ticket_id IS NULL;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT p.passenger_id, p.first_name, p.last_name, p.email
    -> FROM passengers p
    -> LEFT JOIN bookings b       ON b.passenger_id = p.passenger_id
    -> LEFT JOIN ticket_details t ON t.passenger_id = p.passenger_id
    -> WHERE b.booking_id IS NULL AND t.ticket_id IS NULL;
+--------------+------------+-----------+----------------------------+
| passenger_id | first_name | last_name | email                      |
+--------------+------------+-----------+----------------------------+
|           30 | Rahul      | Joshi     | rahul.joshi@example.com    |
|           33 | Priyanka   | Nair      | priyanka.nair23@example.com |
|           46 | Harsha     | Vardhan   | harsha.vardhan@example.com |
+--------------+------------+-----------+----------------------------+
3 rows in set (0.00 sec)
```

What it shows: Two LEFT JOINs: the passenger has neither made a booking nor been a traveller on someone else's booking. These customers are targets for a welcome offer.

9.6 Routes and their return route (self join)

```sql
SELECT o.flight_number AS outbound, o.origin_code, o.destination_code,
       i.flight_number AS inbound, o.scheduled_departure AS out_time, i.scheduled_departure AS back_time
FROM routes o
JOIN routes i ON i.origin_code = o.destination_code
             AND i.destination_code = o.origin_code
WHERE o.origin_code = 'HYD'
ORDER BY o.flight_number;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT o.flight_number AS outbound, o.origin_code, o.destination_code,
    ->        i.flight_number AS inbound, o.scheduled_departure AS out_time, i.scheduled_departure AS back_tim
e
    -> FROM routes o
    -> JOIN routes i ON i.origin_code = o.destination_code
    ->              AND i.destination_code = o.origin_code
    -> WHERE o.origin_code = 'HYD'
    -> ORDER BY o.flight_number;
+----------+-------------+------------------+---------+----------+-----------+
| outbound | origin_code | destination_code | inbound | out_time | back_time |
+----------+-------------+------------------+---------+----------+-----------+
| SW101    | HYD         | DEL              | SW102   | 06:10:00 | 09:30:00  |
| SW201    | HYD         | BOM              | SW202   | 07:40:00 | 10:20:00  |
| SW301    | HYD         | BLR              | SW302   | 08:15:00 | 11:05:00  |
| SW401    | HYD         | MAA              | SW402   | 13:30:00 | 16:10:00  |
| SW701    | HYD         | VTZ              | SW702   | 17:20:00 | 19:10:00  |
| SW801    | HYD         | DXB              | SW802   | 20:30:00 | 03:30:00  |
+----------+-------------+------------------+---------+----------+-----------+
6 rows in set (0.00 sec)
```

What it shows: `routes` is joined to itself: a route matches another whose origin and destination are swapped. Restricting to departures from HYD lists each pair once.

Part 2 – Summary Statistics

9.7 Revenue and traffic per route

```
SELECT r.flight_number, CONCAT(r.origin_code, '-', r.destination_code) AS sector,
       COUNT(DISTINCT f.flight_id)                AS flights_with_pax,
       SUM(t.ticket_status = 'Boarded')           AS boarded,
       SUM(t.fare)                                AS revenue,
       ROUND(AVG(t.fare))                         AS avg_fare
FROM routes r
JOIN flights f        ON f.route_id  = r.route_id
JOIN ticket_details t ON t.flight_id = f.flight_id AND t.ticket_status <> 'Cancelled'
GROUP BY r.route_id, r.flight_number, r.origin_code, r.destination_code
ORDER BY revenue DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT r.flight_number, CONCAT(r.origin_code, '-', r.destination_code) AS sector,
    ->        COUNT(DISTINCT f.flight_id)                AS flights_with_pax,
    ->        SUM(t.ticket_status = 'Boarded')           AS boarded,
    ->        SUM(t.fare)                                AS revenue,
    ->        ROUND(AVG(t.fare))                         AS avg_fare
    -> FROM routes r
    -> JOIN flights f        ON f.route_id  = r.route_id
    -> JOIN ticket_details t ON t.flight_id = f.flight_id AND t.ticket_status <> 'Cancelled'
    -> GROUP BY r.route_id, r.flight_number, r.origin_code, r.destination_code
    -> ORDER BY revenue DESC;
+---------------+---------+------------------+---------+-----------+----------+
| flight_number | sector  | flights_with_pax | boarded | revenue   | avg_fare |
+---------------+---------+------------------+---------+-----------+----------+
| SW901         | HYD-SIN |                3 |       6 | 243115.00 |    30389 |
| SW801         | HYD-DXB |                5 |      12 | 233962.00 |    12998 |
| SW102         | DEL-HYD |                6 |      19 | 182526.00 |     7301 |
| SW101         | HYD-DEL |                5 |      11 | 171394.00 |     9021 |
| SW802         | DXB-HYD |                4 |       8 | 143150.00 |    11012 |
| SW302         | BLR-HYD |                6 |      13 | 133396.00 |     4169 |
| SW202         | BOM-HYD |                6 |      11 | 103080.00 |     5727 |
| SW301         | HYD-BLR |                6 |      12 |  98928.00 |     3957 |
| SW701         | HYD-VTZ |                6 |      15 |  86200.00 |     3592 |
| SW401         | HYD-MAA |                6 |      14 |  79130.00 |     5275 |
| SW501         | HYD-CCU |                6 |       4 |  74604.00 |     9326 |
| SW702         | VTZ-HYD |                5 |      20 |  72410.00 |     3148 |
| SW601         | HYD-GOI |                4 |      12 |  72018.00 |     4801 |
| SW651         | HYD-COK |                4 |       6 |  66948.00 |     6086 |
| SW201         | HYD-BOM |                4 |       7 |  61608.00 |     4107 |
| SW402         | MAA-HYD |                5 |       9 |  44020.00 |     3668 |
+---------------+---------+------------------+---------+-----------+----------+
16 rows in set (0.00 sec)
```

What it shows: Joining routes, flights and tickets and grouping by route gives the network performance table.

9.8 Revenue by cabin on each sector (cross-tab)

```
SELECT CONCAT(r.origin_code, '-', r.destination_code) AS sector,
       SUM(IF(t.class_code = 'EC', t.fare, 0)) AS economy,
       SUM(IF(t.class_code = 'PE', t.fare, 0)) AS premium,
       SUM(IF(t.class_code = 'BU', t.fare, 0)) AS business,
       SUM(IF(t.class_code = 'FC', t.fare, 0)) AS first_class,
       SUM(t.fare)                             AS total
FROM ticket_details t
JOIN flights f ON f.flight_id = t.flight_id
JOIN routes  r ON r.route_id  = f.route_id
WHERE t.ticket_status <> 'Cancelled'
GROUP BY r.origin_code, r.destination_code
ORDER BY total DESC
LIMIT 10;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT CONCAT(r.origin_code, '-', r.destination_code) AS sector,
    ->        SUM(IF(t.class_code = 'EC', t.fare, 0)) AS economy,
    ->        SUM(IF(t.class_code = 'PE', t.fare, 0)) AS premium,
    ->        SUM(IF(t.class_code = 'BU', t.fare, 0)) AS business,
    ->        SUM(IF(t.class_code = 'FC', t.fare, 0)) AS first_class,
    ->        SUM(t.fare)                             AS total
    -> FROM ticket_details t
    -> JOIN flights f ON f.flight_id = t.flight_id
    -> JOIN routes  r ON r.route_id  = f.route_id
    -> WHERE t.ticket_status <> 'Cancelled'
    -> GROUP BY r.origin_code, r.destination_code
    -> ORDER BY total DESC
    -> LIMIT 10;
+---------+-----------+----------+----------+-------------+-----------+
| sector  | economy   | premium  | business | first_class | total     |
+---------+-----------+----------+----------+-------------+-----------+
| HYD-SIN |  69530.00 | 21840.00 |     0.00 |   151745.00 | 243115.00 |
| HYD-DXB | 165210.00 | 32368.00 | 36384.00 |        0.00 | 233962.00 |
| DEL-HYD | 102270.00 | 43296.00 | 36960.00 |        0.00 | 182526.00 |
| HYD-DEL |  88930.00 |     0.00 | 82464.00 |        0.00 | 171394.00 |
| DXB-HYD | 110750.00 | 32400.00 |     0.00 |        0.00 | 143150.00 |
| BLR-HYD |  99700.00 |     0.00 | 33696.00 |        0.00 | 133396.00 |
| BOM-HYD |  51480.00 | 12272.00 | 39328.00 |        0.00 | 103080.00 |
| HYD-BLR |  88560.00 |     0.00 | 10368.00 |        0.00 |  98928.00 |
| HYD-VTZ |  86200.00 |     0.00 |     0.00 |        0.00 |  86200.00 |
| HYD-MAA |  32410.00 | 22720.00 | 24000.00 |        0.00 |  79130.00 |
+---------+-----------+----------+----------+-------------+-----------+
10 rows in set (0.00 sec)
```

What it shows: Conditional sums turn the four cabins into columns. First Class only appears on the 787 routes.

9.9 Revenue by destination country and city with subtotals (WITH ROLLUP)

```
SELECT IF(GROUPING(country), 'ALL COUNTRIES', country) AS country,
       IF(GROUPING(city), IF(GROUPING(country), '', '(subtotal)'), city) AS city,
       COUNT(*) AS tickets, SUM(fare) AS revenue
FROM (SELECT a.country, a.city, t.fare
      FROM ticket_details t
      JOIN flights  f ON f.flight_id      = t.flight_id
      JOIN routes   r ON r.route_id       = f.route_id
      JOIN airports a ON a.airport_code   = r.destination_code
      WHERE t.ticket_status <> 'Cancelled') AS x
GROUP BY country, city WITH ROLLUP;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT IF(GROUPING(country), 'ALL COUNTRIES', country) AS country,
    ->        IF(GROUPING(city), IF(GROUPING(country), '', '(subtotal)'), city) AS city,
    ->        COUNT(*) AS tickets, SUM(fare) AS revenue
    -> FROM (SELECT a.country, a.city, t.fare
    ->       FROM ticket_details t
    ->       JOIN flights  f ON f.flight_id      = t.flight_id
    ->       JOIN routes   r ON r.route_id       = f.route_id
    ->       JOIN airports a ON a.airport_code   = r.destination_code
    ->       WHERE t.ticket_status <> 'Cancelled') AS x
    -> GROUP BY country, city WITH ROLLUP;
+----------------------+---------------+---------+------------+
| country              | city          | tickets | revenue    |
+----------------------+---------------+---------+------------+
| India                | Bengaluru     |      25 |   98928.00 |
| India                | Chennai       |      15 |   79130.00 |
| India                | Goa           |      15 |   72018.00 |
| India                | Hyderabad     |     123 |  678582.00 |
| India                | Kochi         |      11 |   66948.00 |
| India                | Kolkata       |       8 |   74604.00 |
| India                | Mumbai        |      15 |   61608.00 |
| India                | New Delhi     |      19 |  171394.00 |
| India                | Visakhapatnam |      24 |   86200.00 |
| India                | (subtotal)    |     255 | 1389412.00 |
| Singapore            | Singapore     |       8 |  243115.00 |
| Singapore            | (subtotal)    |       8 |  243115.00 |
| United Arab Emirates | Dubai         |      18 |  233962.00 |
| United Arab Emirates | (subtotal)    |      18 |  233962.00 |
| ALL COUNTRIES        |               |     281 | 1866489.00 |
+----------------------+---------------+---------+------------+
15 rows in set, 2 warnings (0.00 sec)
```

What it shows: ROLLUP adds a subtotal per country and a grand total. `GROUPING()` labels those rows. The subquery in FROM gathers the destination of every live ticket.

9.10 Ranking routes by revenue within domestic and international (window function)

```sql
SELECT IF(orig.country = 'India' AND dest.country = 'India', 'Domestic', 'International') AS market,
       r.flight_number, SUM(t.fare) AS revenue,
       RANK() OVER (PARTITION BY IF(orig.country = 'India' AND dest.country = 'India', 'Domestic', 'International')
                    ORDER BY SUM(t.fare) DESC) AS rank_in_market
FROM ticket_details t
JOIN flights  f    ON f.flight_id      = t.flight_id
JOIN routes   r    ON r.route_id       = f.route_id
JOIN airports orig ON orig.airport_code = r.origin_code
JOIN airports dest ON dest.airport_code = r.destination_code
WHERE t.ticket_status <> 'Cancelled'
GROUP BY market, r.flight_number, dest.country, orig.country
ORDER BY market, rank_in_market;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT IF(orig.country = 'India' AND dest.country = 'India', 'Domestic', 'International') AS market,
    ->        r.flight_number, SUM(t.fare) AS revenue,
    ->        RANK() OVER (PARTITION BY IF(orig.country = 'India' AND dest.country = 'India', 'Domestic', 'Int
ernational')
    ->                     ORDER BY SUM(t.fare) DESC) AS rank_in_market
    -> FROM ticket_details t
    -> JOIN flights  f    ON f.flight_id      = t.flight_id
    -> JOIN routes   r    ON r.route_id       = f.route_id
    -> JOIN airports orig ON orig.airport_code = r.origin_code
    -> JOIN airports dest ON dest.airport_code = r.destination_code
    -> WHERE t.ticket_status <> 'Cancelled'
    -> GROUP BY market, r.flight_number, dest.country, orig.country
    -> ORDER BY market, rank_in_market;
+---------------+---------------+-----------+----------------+
| market        | flight_number | revenue   | rank_in_market |
+---------------+---------------+-----------+----------------+
| Domestic      | SW102         | 182526.00 |              1 |
| Domestic      | SW101         | 171394.00 |              2 |
| Domestic      | SW302         | 133396.00 |              3 |
| Domestic      | SW202         | 103080.00 |              4 |
| Domestic      | SW301         |  98928.00 |              5 |
| Domestic      | SW701         |  86200.00 |              6 |
| Domestic      | SW401         |  79130.00 |              7 |
| Domestic      | SW501         |  74604.00 |              8 |
| Domestic      | SW702         |  72410.00 |              9 |
| Domestic      | SW601         |  72018.00 |             10 |
| Domestic      | SW651         |  66948.00 |             11 |
| Domestic      | SW201         |  61608.00 |             12 |
| Domestic      | SW402         |  44020.00 |             13 |
| International | SW901         | 243115.00 |              1 |
| International | SW801         | 233962.00 |              2 |
| International | SW802         | 143150.00 |              3 |
+---------------+---------------+-----------+----------------+
16 rows in set (0.00 sec)
```

What it shows: A route is domestic only if both of its airports are in India. `PARTITION BY` restarts the ranking for each market, and `airports` is joined twice under two aliases (origin and destination).

Part 3 – Subquery Reports

9.11 Bookings worth more than the average booking (scalar subquery)

```sql
SELECT booking_id, booking_reference, active_tickets, total_fare
FROM v_booking_totals
WHERE status = 'Confirmed'
  AND total_fare > (SELECT AVG(total_fare) FROM v_booking_totals WHERE status = 'Confirmed')
ORDER BY total_fare DESC
LIMIT 10;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT booking_id, booking_reference, active_tickets, total_fare
    -> FROM v_booking_totals
    -> WHERE status = 'Confirmed'
    ->   AND total_fare > (SELECT AVG(total_fare) FROM v_booking_totals WHERE status = 'Confirmed')
    -> ORDER BY total_fare DESC
    -> LIMIT 10;
+------------+-------------------+----------------+------------+
| booking_id | booking_reference | active_tickets | total_fare |
+------------+-------------------+----------------+------------+
|         45 | HW4BFV            |              1 |   76670.00 |
|         87 | 8VHJMF            |              1 |   75075.00 |
|          3 | 4PVEGW            |              3 |   62592.00 |
|         36 | JT66LL            |              4 |   46120.00 |
|         27 | 8CEQ8J            |              4 |   40040.00 |
|        144 | F47NXW            |              1 |   36384.00 |
|         34 | ZV2HDJ            |              4 |   33408.00 |
|        157 | MRLKD2            |              3 |   30780.00 |
|         58 | XGYQUY            |              2 |   30592.00 |
|        123 | K3N278            |              3 |   30540.00 |
+------------+-------------------+----------------+------------+
10 rows in set (0.02 sec)
```

What it shows: The inner query returns one number, the average confirmed booking value, which every booking is compared with.

9.12 Passengers who have flown internationally (nested IN)

```sql
SELECT passenger_id, first_name, last_name, nationality
FROM passengers
WHERE passenger_id IN (
    SELECT t.passenger_id FROM ticket_details t
    WHERE t.ticket_status = 'Boarded'
      AND t.flight_id IN (
          SELECT f.flight_id FROM flights f
          WHERE f.route_id IN (
              SELECT r.route_id FROM routes r
              JOIN airports a ON a.airport_code = r.destination_code
                             OR a.airport_code = r.origin_code
              WHERE a.country <> 'India')))
ORDER BY last_name, first_name;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT passenger_id, first_name, last_name, nationality
    -> FROM passengers
    -> WHERE passenger_id IN (
    ->     SELECT t.passenger_id FROM ticket_details t
    ->     WHERE t.ticket_status = 'Boarded'
    ->       AND t.flight_id IN (
    ->           SELECT f.flight_id FROM flights f
    ->           WHERE f.route_id IN (
    ->               SELECT r.route_id FROM routes r
    ->               JOIN airports a ON a.airport_code = r.destination_code
    ->                              OR a.airport_code = r.origin_code
    ->               WHERE a.country <> 'India')))
    -> ORDER BY last_name, first_name;
+--------------+------------+-------------+-------------+
| passenger_id | first_name | last_name   | nationality |
+--------------+------------+-------------+-------------+
|           44 | Omar       | Al Mansoori | Emirati     |
|           39 | Shreya     | Banerjee    | Indian      |
|           26 | Sai        | Charan      | Indian      |
|           37 | Bhavya     | Goud        | Indian      |
|           15 | Zara       | Khan        | Indian      |
|           41 | Pallavi    | Kulkarni    | Indian      |
|           10 | Akhil      | Naidu       | Indian      |
|           16 | Nikhil     | Patel       | Indian      |
|           18 | Prakash    | Rao         | Indian      |
|           21 | Rohith     | Rao         | Indian      |
|           20 | Sahithi    | Rao         | Indian      |
|           19 | Sunitha    | Rao         | Indian      |
|            3 | Harika     | Reddy       | Indian      |
|            2 | Lakshmi    | Reddy       | Indian      |
|            1 | Srinivas   | Reddy       | Indian      |
|            4 | Varun      | Reddy       | Indian      |
|            7 | Aryan      | Sharma      | Indian      |
|            6 | Pooja      | Sharma      | Indian      |
|            5 | Rajesh     | Sharma      | Indian      |
|           42 | James      | Wilson      | British     |
|           38 | Naveen     | Yadav       | Indian      |
+--------------+------------+-------------+-------------+
21 rows in set (0.00 sec)
```

What it shows: Three nested subqueries work from foreign airports to routes, from routes to flights, and from flights to boarded tickets. The outer query lists each traveller once.

9.13 Each traveller's most recent completed flight (correlated subquery)

```sql
SELECT CONCAT(p.first_name, ' ', p.last_name) AS traveller, r.flight_number,
       r.origin_code, r.destination_code, f.departure_time
FROM ticket_details t
JOIN passengers p ON p.passenger_id = t.passenger_id
JOIN flights    f ON f.flight_id    = t.flight_id
JOIN routes     r ON r.route_id     = f.route_id
WHERE t.ticket_status = 'Boarded'
  AND f.departure_time = (SELECT MAX(f2.departure_time)
                          FROM ticket_details t2
                          JOIN flights f2 ON f2.flight_id = t2.flight_id
                          WHERE t2.passenger_id = t.passenger_id
                            AND t2.ticket_status = 'Boarded')
ORDER BY f.departure_time DESC
LIMIT 15;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT CONCAT(p.first_name, ' ', p.last_name) AS traveller, r.flight_number,
    ->        r.origin_code, r.destination_code, f.departure_time
    -> FROM ticket_details t
    -> JOIN passengers p ON p.passenger_id = t.passenger_id
    -> JOIN flights    f ON f.flight_id    = t.flight_id
    -> JOIN routes     r ON r.route_id     = f.route_id
    -> WHERE t.ticket_status = 'Boarded'
    ->   AND f.departure_time = (SELECT MAX(f2.departure_time)
    ->                           FROM ticket_details t2
    ->                           JOIN flights f2 ON f2.flight_id = t2.flight_id
    ->                           WHERE t2.passenger_id = t.passenger_id
    ->                             AND t2.ticket_status = 'Boarded')
    -> ORDER BY f.departure_time DESC
    -> LIMIT 15;
+------------------+---------------+-------------+------------------+---------------------+
| traveller        | flight_number | origin_code | destination_code | departure_time      |
+------------------+---------------+-------------+------------------+---------------------+
| Imran Khan       | SW101         | HYD         | DEL              | 2026-10-25 06:10:00 |
| Ayesha Khan      | SW101         | HYD         | DEL              | 2026-10-25 06:10:00 |
| Zara Khan        | SW101         | HYD         | DEL              | 2026-10-25 06:10:00 |
| Sneha Desai      | SW401         | HYD         | MAA              | 2026-10-24 13:30:00 |
| Arjun Malhotra   | SW401         | HYD         | MAA              | 2026-10-24 13:30:00 |
| Sahithi Rao      | SW302         | BLR         | HYD              | 2026-10-23 11:05:00 |
| James Wilson     | SW302         | BLR         | HYD              | 2026-10-23 11:05:00 |
| Aryan Sharma     | SW702         | VTZ         | HYD              | 2026-10-22 19:10:00 |
| Kiran Kumar      | SW702         | VTZ         | HYD              | 2026-10-22 19:10:00 |
| Rajesh Sharma    | SW702         | VTZ         | HYD              | 2026-10-22 19:10:00 |
| Pooja Sharma     | SW702         | VTZ         | HYD              | 2026-10-22 19:10:00 |
| Manoj Tiwari     | SW701         | HYD         | VTZ              | 2026-10-21 17:20:00 |
| Prakash Rao      | SW202         | BOM         | HYD              | 2026-10-21 10:20:00 |
| Sunitha Rao      | SW202         | BOM         | HYD              | 2026-10-21 10:20:00 |
| Pallavi Kulkarni | SW201         | HYD         | BOM              | 2026-10-20 07:40:00 |
+------------------+---------------+-------------+------------------+---------------------+
15 rows in set (0.03 sec)
```

What it shows: The subquery depends on the outer row's passenger and returns that passenger's last boarded departure.

9.14 Customers who booked but have never actually flown (NOT EXISTS)

```sql
SELECT p.passenger_id, p.first_name, p.last_name,
       (SELECT COUNT(*) FROM bookings b WHERE b.passenger_id = p.passenger_id) AS bookings_made
FROM passengers p
WHERE EXISTS (SELECT 1 FROM bookings b WHERE b.passenger_id = p.passenger_id)
  AND NOT EXISTS (SELECT 1 FROM ticket_details t
                  WHERE t.passenger_id = p.passenger_id AND t.ticket_status = 'Boarded');
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT p.passenger_id, p.first_name, p.last_name,
    ->        (SELECT COUNT(*) FROM bookings b WHERE b.passenger_id = p.passenger_id) AS bookings_made
    -> FROM passengers p
    -> WHERE EXISTS (SELECT 1 FROM bookings b WHERE b.passenger_id = p.passenger_id)
    ->   AND NOT EXISTS (SELECT 1 FROM ticket_details t
    ->                   WHERE t.passenger_id = p.passenger_id AND t.ticket_status = 'Boarded');
+--------------+------------+-----------+---------------+
| passenger_id | first_name | last_name | bookings_made |
+--------------+------------+-----------+---------------+
|           23 | Ananya     | Mehta     |             2 |
|           25 | Deepika    | Varma     |             2 |
+--------------+------------+-----------+---------------+
2 rows in set (0.00 sec)
```

What it shows: EXISTS and NOT EXISTS combined: the person has made a booking, but no ticket of theirs was ever boarded (their trips are in the future, cancelled or no-shows).

9.15 Bookings that needed a retry: a failed attempt followed by a success (EXISTS)

```
SELECT b.booking_reference,
       (SELECT GROUP_CONCAT(p.payment_method, ':', p.payment_status ORDER BY p.transaction_date SEPARATOR ' -> ')
        FROM payments p WHERE p.booking_id = b.booking_id) AS attempts,
       (SELECT amount FROM payments p WHERE p.booking_id = b.booking_id
                                      AND p.payment_status = 'Success') AS paid
FROM bookings b
WHERE EXISTS (SELECT 1 FROM payments p WHERE p.booking_id = b.booking_id AND p.payment_status = 'Failed')
  AND EXISTS (SELECT 1 FROM payments p WHERE p.booking_id = b.booking_id AND p.payment_status = 'Success')
ORDER BY b.booking_id
LIMIT 12;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT b.booking_reference,
    ->        (SELECT GROUP_CONCAT(p.payment_method, ':', p.payment_status ORDER BY p.transaction_date SEPARAT
OR ' -> ')
    ->         FROM payments p WHERE p.booking_id = b.booking_id) AS attempts,
    ->        (SELECT amount FROM payments p WHERE p.booking_id = b.booking_id
    ->                                       AND p.payment_status = 'Success') AS paid
    -> FROM bookings b
    -> WHERE EXISTS (SELECT 1 FROM payments p WHERE p.booking_id = b.booking_id AND p.payment_status = 'Failed
')
    ->   AND EXISTS (SELECT 1 FROM payments p WHERE p.booking_id = b.booking_id AND p.payment_status = 'Succes
s')
    -> ORDER BY b.booking_id
    -> LIMIT 12;
+-------------------+-----------------------------------------+----------+
| booking_reference | attempts                                | paid     |
+-------------------+-----------------------------------------+----------+
| 4PVEGW            | Debit Card:Failed -> Credit Card:Success  | 62592.00 |
| 52QQC2            | Credit Card:Failed -> UPI:Success         |  4250.00 |
| 5C33HK            | Credit Card:Failed -> UPI:Success         |  3330.00 |
| 8Z7EP5            | Debit Card:Failed -> Credit Card:Success  | 15520.00 |
| SZUENU            | Credit Card:Failed -> UPI:Success         |  5248.00 |
| CMRAY2            | Debit Card:Failed -> Credit Card:Success  |  4070.00 |
| S6SNJB            | Debit Card:Failed -> UPI:Success          |  3260.00 |
| 6NGK3H            | Credit Card:Failed -> Debit Card:Success  |  6210.00 |
| BXHXYM            | Credit Card:Failed -> Debit Card:Success  | 29680.00 |
| KJ5AS9            | Credit Card:Failed -> Credit Card:Success |  4850.00 |
| 4QEU64            | Credit Card:Failed -> UPI:Success         |  4190.00 |
| BJ6XPV            | Debit Card:Failed -> Debit Card:Success   |  6500.00 |
+-------------------+-----------------------------------------+----------+
12 rows in set (0.01 sec)
```

What it shows: Two correlated EXISTS tests select the bookings; correlated subqueries in SELECT show the attempt history. This report was impossible with the original 1 : 1 payments design.

9.16 Customer value segments by total spend (derived table + CASE)

```
SELECT segment, COUNT(*) AS customers, ROUND(AVG(spend)) AS avg_spend, SUM(spend) AS total_spend
FROM (SELECT b.passenger_id, SUM(p.amount) AS spend,
             CASE WHEN SUM(p.amount) >= 60000 THEN 'Platinum'
                  WHEN SUM(p.amount) >= 25000 THEN 'Gold'
                  ELSE 'Silver' END AS segment
      FROM bookings b
      JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Success'
      GROUP BY b.passenger_id) AS spend_per_customer
GROUP BY segment
ORDER BY total_spend DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT segment, COUNT(*) AS customers, ROUND(AVG(spend)) AS avg_spend, SUM(spend) AS total_spend
    -> FROM (SELECT b.passenger_id, SUM(p.amount) AS spend,
    ->              CASE WHEN SUM(p.amount) >= 60000 THEN 'Platinum'
    ->                   WHEN SUM(p.amount) >= 25000 THEN 'Gold'
    ->                   ELSE 'Silver' END AS segment
    ->       FROM bookings b
    ->       JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Success'
    ->       GROUP BY b.passenger_id) AS spend_per_customer
    -> GROUP BY segment
    -> ORDER BY total_spend DESC;
+----------+-----------+-----------+-------------+
| segment  | customers | avg_spend | total_spend |
+----------+-----------+-----------+-------------+
| Platinum |         8 |    106038 |   848307.00 |
| Gold     |        21 |     38181 |   801804.00 |
| Silver   |        14 |     14462 |   202466.00 |
+----------+-----------+-----------+-------------+
3 rows in set (0.00 sec)
```

What it shows: The derived table totals each customer's successful payments and assigns a segment; the outer query summarizes the segments.

9.17 Daily ticket revenue in October with a running total (CTE + SUM OVER)

```sql
WITH daily AS (
    SELECT DATE(f.departure_time) AS flight_day, SUM(t.fare) AS revenue
    FROM ticket_details t
    JOIN flights f ON f.flight_id = t.flight_id
    WHERE t.ticket_status IN ('Boarded', 'No-show')
      AND f.departure_time >= '2026-10-01'
    GROUP BY DATE(f.departure_time)
)
SELECT flight_day, revenue,
       SUM(revenue) OVER (ORDER BY flight_day) AS running_total
FROM daily
ORDER BY flight_day;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> WITH daily AS (
    ->     SELECT DATE(f.departure_time) AS flight_day, SUM(t.fare) AS revenue
    ->     FROM ticket_details t
    ->     JOIN flights f ON f.flight_id = t.flight_id
    ->     WHERE t.ticket_status IN ('Boarded', 'No-show')
    ->       AND f.departure_time >= '2026-10-01'
    ->     GROUP BY DATE(f.departure_time)
    -> )
    -> SELECT flight_day, revenue,
    ->        SUM(revenue) OVER (ORDER BY flight_day) AS running_total
    -> FROM daily
    -> ORDER BY flight_day;
+------------+-----------+---------------+
| flight_day | revenue   | running_total |
+------------+-----------+---------------+
| 2026-10-01 |  48410.00 |      48410.00 |
| 2026-10-02 | 146005.00 |     194415.00 |
| 2026-10-03 |   3200.00 |     197615.00 |
| 2026-10-04 |  35840.00 |     233455.00 |
| 2026-10-05 |  71818.00 |     305273.00 |
| 2026-10-06 |  74328.00 |     379601.00 |
| 2026-10-07 |  18340.00 |     397941.00 |
| 2026-10-08 |  20850.00 |     418791.00 |
| 2026-10-09 |  95510.00 |     514301.00 |
| 2026-10-10 |  51216.00 |     565517.00 |
| 2026-10-11 |  95450.00 |     660967.00 |
| 2026-10-12 |  68308.00 |     729275.00 |
| 2026-10-13 |   3300.00 |     732575.00 |
| 2026-10-14 |  28350.00 |     760925.00 |
| 2026-10-15 |  17960.00 |     778885.00 |
| 2026-10-16 |  13040.00 |     791925.00 |
| 2026-10-17 |   5248.00 |     797173.00 |
| 2026-10-18 |  40040.00 |     837213.00 |
| 2026-10-19 |  70724.00 |     907937.00 |
| 2026-10-20 |   4230.00 |     912167.00 |
| 2026-10-21 |  29600.00 |     941767.00 |
| 2026-10-22 |  23520.00 |     965287.00 |
| 2026-10-23 |   7540.00 |     972827.00 |
| 2026-10-24 |  14070.00 |     986897.00 |
| 2026-10-25 |  62592.00 |    1049489.00 |
+------------+-----------+---------------+
25 rows in set (0.00 sec)
```

What it shows: The CTE computes revenue earned per day of travel; the window function accumulates it into a month-to-date total. No-show tickets count because their fare is not refunded.

9.18 Fleet utilization: flights and block hours per aircraft (LEFT JOIN + derived values)

```sql
SELECT a.registration, a.model, a.status,
       COUNT(f.flight_id)                                                    AS flights_flown,
       ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, f.departure_time, f.arrival_time)), 0) / 60, 1)
                                                                             AS block_hours
FROM aircraft a
LEFT JOIN flights f ON f.aircraft_id = a.aircraft_id AND f.flight_status = 'Arrived'
GROUP BY a.aircraft_id, a.registration, a.model, a.status
ORDER BY block_hours DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT a.registration, a.model, a.status,
    ->        COUNT(f.flight_id)                                                    AS flights_flown,
    ->        ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, f.departure_time, f.arrival_time)), 0) / 60, 1)
    ->                                                                              AS block_hours
    -> FROM aircraft a
    -> LEFT JOIN flights f ON f.aircraft_id = a.aircraft_id AND f.flight_status = 'Arrived'
    -> GROUP BY a.aircraft_id, a.registration, a.model, a.status
    -> ORDER BY block_hours DESC;
+--------------+----------------+-------------+---------------+-------------+
| registration | model          | status      | flights_flown | block_hours |
+--------------+----------------+-------------+---------------+-------------+
| VT-SWF       | Boeing 787-9   | Active      |             9 |        38.8 |
| VT-SWA       | Airbus A320neo | Active      |             9 |        19.8 |
| VT-SWC       | Airbus A321neo | Active      |            13 |        18.5 |
| VT-SWD       | Boeing 737-8   | Active      |            12 |        15.3 |
| VT-SWB       | Airbus A320neo | Active      |            10 |        14.5 |
| VT-SWE       | ATR 72-600     | Active      |             8 |         9.3 |
| VT-SWG       | Airbus A320neo | Maintenance |             0 |         0.0 |
+--------------+----------------+-------------+---------------+-------------+
7 rows in set (0.00 sec)
```

What it shows: The LEFT JOIN keeps the aircraft in maintenance (zero hours); the flight-status filter sits in ON for the same reason.

9.19 Refund report: why each cancelled booking was refunded

```sql
SELECT b.booking_reference, b.status, p.amount AS refunded,
       CASE WHEN EXISTS (SELECT 1 FROM ticket_details t
                         JOIN flights f ON f.flight_id = t.flight_id
                         WHERE t.booking_id = b.booking_id AND f.flight_status = 'Cancelled')
            THEN 'Flight cancelled by airline'
            ELSE 'Cancelled by passenger' END AS reason
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Refunded'
ORDER BY reason, refunded DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT b.booking_reference, b.status, p.amount AS refunded,
    ->        CASE WHEN EXISTS (SELECT 1 FROM ticket_details t
    ->                          JOIN flights f ON f.flight_id = t.flight_id
    ->                          WHERE t.booking_id = b.booking_id AND f.flight_status = 'Cancelled')
    ->             THEN 'Flight cancelled by airline'
    ->             ELSE 'Cancelled by passenger' END AS reason
    -> FROM bookings b
    -> JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Refunded'
    -> ORDER BY reason, refunded DESC;
+-------------------+-----------+----------+----------------------------+
| booking_reference | status    | refunded | reason                     |
+-------------------+-----------+----------+----------------------------+
| 9CCB25            | Cancelled | 50120.00 | Cancelled by passenger     |
| JDXKMZ            | Cancelled | 19740.00 | Cancelled by passenger     |
| LFXLX6            | Cancelled | 13200.00 | Cancelled by passenger     |
| T3YEBR            | Cancelled | 11610.00 | Cancelled by passenger     |
| FTM3C9            | Cancelled | 11460.00 | Cancelled by passenger     |
| ET3B53            | Cancelled |  6560.00 | Cancelled by passenger     |
| S7N7UT            | Cancelled |  5800.00 | Cancelled by passenger     |
| 5H9ET7            | Cancelled |  5750.00 | Cancelled by passenger     |
| BKMYG2            | Cancelled |  5750.00 | Cancelled by passenger     |
| MLVJUD            | Cancelled |  5260.00 | Cancelled by passenger     |
| K4YYCJ            | Cancelled |  3300.00 | Cancelled by passenger     |
| RL2F4N            | Cancelled |  3260.00 | Cancelled by passenger     |
| ASKWCP            | Cancelled | 13312.00 | Flight cancelled by airline |
| LUEWGH            | Cancelled |  4160.00 | Flight cancelled by airline |
| NB9GJZ            | Cancelled |  4160.00 | Flight cancelled by airline |
| UF88YX            | Cancelled |  4160.00 | Flight cancelled by airline |
+-------------------+-----------+----------+----------------------------+
16 rows in set (0.00 sec)
```

What it shows: A correlated EXISTS inside CASE decides whether the airline caused the refund (one of the two cancelled SW651 flights) or the passenger did.

Conclusion

The finalized airline design spreads the data over nine tables, and joins reassemble it into the documents the airline needs: manifests, itineraries, load-factor and revenue reports. LEFT JOINs keep empty flights, idle aircraft and inactive customers in the results, and the views supply booking totals and occupancy without redundant columns. Subqueries answer comparative and existence questions: above average, most recent, booked but never flown, paid after a retry, refunded because of a cancellation. Several of these reports, such as manifests by traveller, payment retries and capacity by aircraft, are possible only because of the improvements made to the original design in Assignment 3.